

Sous le capot du protocole MCP

Codons un serveur en PHP pur

Amaury Bouchard

Forum PHP 2026 · AFUP

1

Pourquoi un talk MCP en 2026

Un standard qui a deux ans, un écosystème PHP qui a déjà changé deux fois.

MCP : standard de facto en un an

novembre 2024

Anthropic publie MCP

protocole ouvert pour brancher
des outils aux LLM

2025

Adoption générale

OpenAI, Google, Microsoft...
tous clients MCP

juillet 2026

MCP devient stateless

révision 2026-07-28 : plus de
handshake ni de session

Le « USB-C des applications LLM » : un connecteur unique entre les modèles et vos données.

Côté PHP, tout a bougé depuis le Forum PHP 2025



mcp-server-bundle (Edouard Courty, Forum PHP 2025)

Le bundle présenté l'an dernier est officiellement déprécié.



symfony/mcp-bundle

Symfony a publié son bundle officiel.

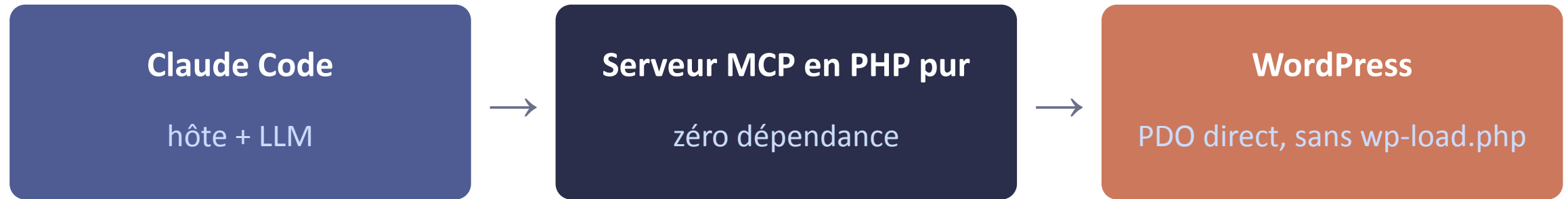


mcp/sdk (PHP Foundation + Symfony)

SDK PHP officiel, framework-agnostic, en cours de stabilisation.

On va regarder ce qu'il se passe à l'intérieur.

Ce qu'on va construire



Le cœur du protocole : moins de 300 lignes de PHP

2

Le protocole MCP

Ce qu'un serveur sait faire

La plomberie : JSON-RPC et transport

Les *tools* : des actions

TOOLS

À quoi ça sert

Exposer des opérations au modèle : lire, chercher, modifier, envoyer. Le LLM les enchaîne librement pour accomplir ce qu'on lui demande, et lit leurs résultats pour décider de la suite.

Qui décide de s'en servir ?

Le LLM. Il choisit le bon *tool* d'après son nom, sa description et son schéma de paramètres.

Comment ça s'utilise

On parle normalement. Pas de syntaxe : la demande suffit à déclencher les bons appels.

```
« Termine mon brouillon sur la pâte à choux »
```

Exemples

- `search_posts(query, status, limit)` : chercher, lister les brouillons ou les articles récents
- `get_post(id)`, `update_post(id, content)` : lire et modifier un article
- ailleurs : `send_email()`, `create_ticket()`, `run_query()`...

Les *resources* : des données

RESOURCES

À quoi ça sert

Rendre des données adressables par une URI : articles, fichiers, schémas, états. On les injecte dans le contexte pour donner au modèle de la matière à lire, sans passer par un tool.

Qui décide de s'en servir ?

L'utilisateur (ou l'application hôte) : c'est lui qui attache la *resource* à la conversation, de façon déterministe.

Comment ça s'utilise

Dans Claude Code, on référence une *resource* avec @, comme un fichier :

```
« Résumé @blog:wp://post/101 »
```

Exemples

- wp://post/{id} : un article du blog (template d'URI, une déclaration pour tous les articles)
- ailleurs : file:///..., schema://ma-base, logs://prod

Les *prompts* : des instructions prêtes à l'emploi

PROMPTS

À quoi ça sert

Encapsuler un savoir-faire derrière un nom et des arguments : les bonnes consignes, et les bons tools à enchaîner. L'auteur du serveur écrit le mode d'emploi une fois, l'utilisateur le rejoue à volonté. Le serveur ne fait qu'en retourner le texte : l'intelligence reste côté LLM.

Qui décide de s'en servir ?

L'utilisateur, explicitement. Un prompt ne se déclenche jamais tout seul.

Comment ça s'utilise

Dans Claude Code, chaque *prompt* devient une commande slash :

```
/mcp__blog__proofread 103
```

Exemples

- `proofread(post_id)` : relecture typographique selon les règles de la maison
- `code_review(merge_request)` : la checklist de revue de l'équipe
- `daily_report(date)` : le rapport quotidien, toujours au même format

Les notifications : des messages sans réponse

CLIENT → SERVEUR

un POST comme les autres

La notification arrive sur POST /mcp, comme toute requête, mais **sans id**. Le serveur accuse réception : HTTP 202, corps vide.

Exemple : notifications/cancelled, l'utilisateur a interrompu, inutile de continuer à travailler.

C'est le sens que notre serveur gère (3 lignes de code).

SERVEUR → CLIENT

uniquement sur un flux SSE ouvert par le client

Le serveur ne peut pas appeler le client. Il ne peut qu'écrire sur un flux que le client a ouvert : le flux SSE de la réponse d'une requête en cours, ou le flux d'abonnement subscriptions/listen.

Exemples : notifications/progress (travail long), listes de tools ou ressources modifiées.

Non émis par notre serveur.

Dans les deux cas, le message est identique : du JSON-RPC sans id. Seul le canal change.

L'enveloppe : JSON-RPC 2.0

```
JSON-RPC.json

// Requête (Claude → serveur)
{
  "jsonrpc": "2.0",
  "id": 42,
  "method": "tools/call",
  "params": {
    "name": "get_post",
    "arguments": {"id": 103}
  }
}
```

- Un id corréle requête et réponse : le transport peut être asynchrone.
- Sans id, c'est une notification : personne n'attend de réponse.

```
JSON-RPC.json

// Réponse (serveur → Claude)
{
  "jsonrpc": "2.0",
  "id": 42,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "{\"ID\":103,\"post_title\":\"Risotto...\"}"
      }
    ],
    "isError": false,
    "resultType": "complete"
  }
}
```

La requête MCP autoportante

La requête MCP autoportante

```
JSON-RPC.json

POST /mcp HTTP/1.1
Content-Type: application/json
MCP-Protocol-Version: 2026-07-28
Mcp-Method: tools/call
Mcp-Name: get_post

{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "get_post",
    "arguments": {"id": 103},
    "_meta": {
      "io.modelcontextprotocol/protocolVersion": "2026-07-28",
      "io.modelcontextprotocol/clientInfo": {
        "name": "claude-code",
        "version": "1.x"
      }
    }
  }
}
```

À retenir

- La requête contient tout : version, identité et capacités voyagent dans _meta.
- Les en-têtes Mcp-* reprennent méthode et nom : les intermédiaires peuvent router sans lire le corps.
- Pas de session : le serveur peut oublier le client entre deux requêtes.

Trois façons de se parler

STDIO

le sous-processus

Le client lance le serveur et lui parle sur stdin/stdout. Aucun réseau, un serveur par client.

Idéal pour un outil local (accès aux fichiers, CLI).

HTTP · RÉPONSE JSON

notre choix

Un POST autoportant sur /mcp, une réponse JSON, terminé.
Sans état : mutualisable derrière un load balancer.

Le mode le plus simple du Streamable HTTP... et taillé pour PHP.

HTTP · RÉPONSE SSE

l'option temps réel

Même endpoint, mais le serveur répond en text/event-stream : il peut pousser des notifications (progression...) avant le résultat.

Le canal des notifications serveur vers client. Optionnel : non implémenté ici.

L'authentification : trois niveaux

0

Aucune : réservé au localhost

Notre démo : serveur sur 127.0.0.1. Même en local, on vérifie l'en-tête Origin (DNS rebinding).

1

Bearer token : le secret partagé

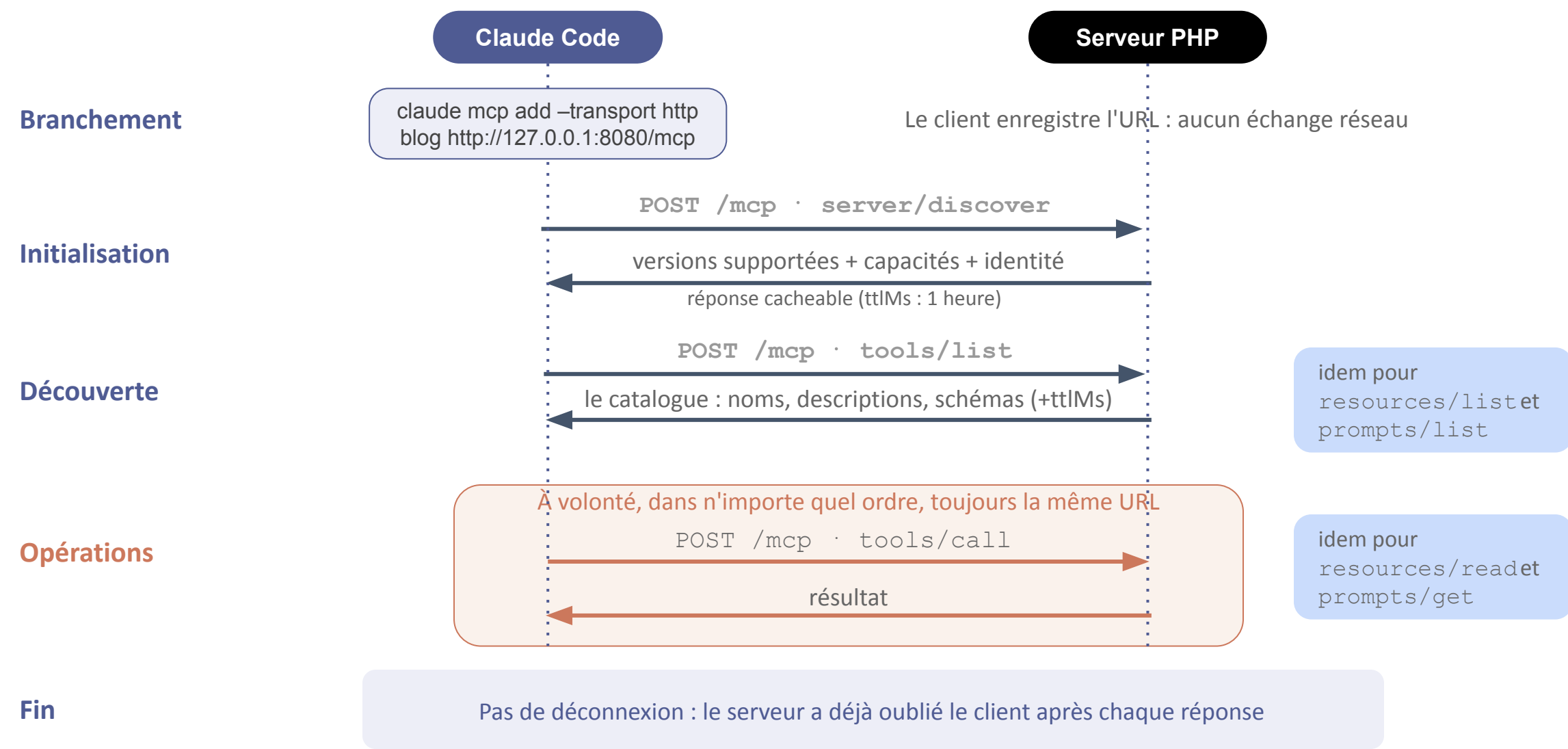
`Authorization: Bearer <jeton>`, comparé avec `hash_equals`. Pour un serveur interne.
Chez nous : 7 lignes, une clé de config.

2

OAuth 2.1 : obligatoire en exposition publique

La spec l'impose pour un serveur HTTP public : serveur d'autorisation, consentement, scopes, jetons.

Le cycle de vie : du branchement à l'oubli



La discovery : tools/list

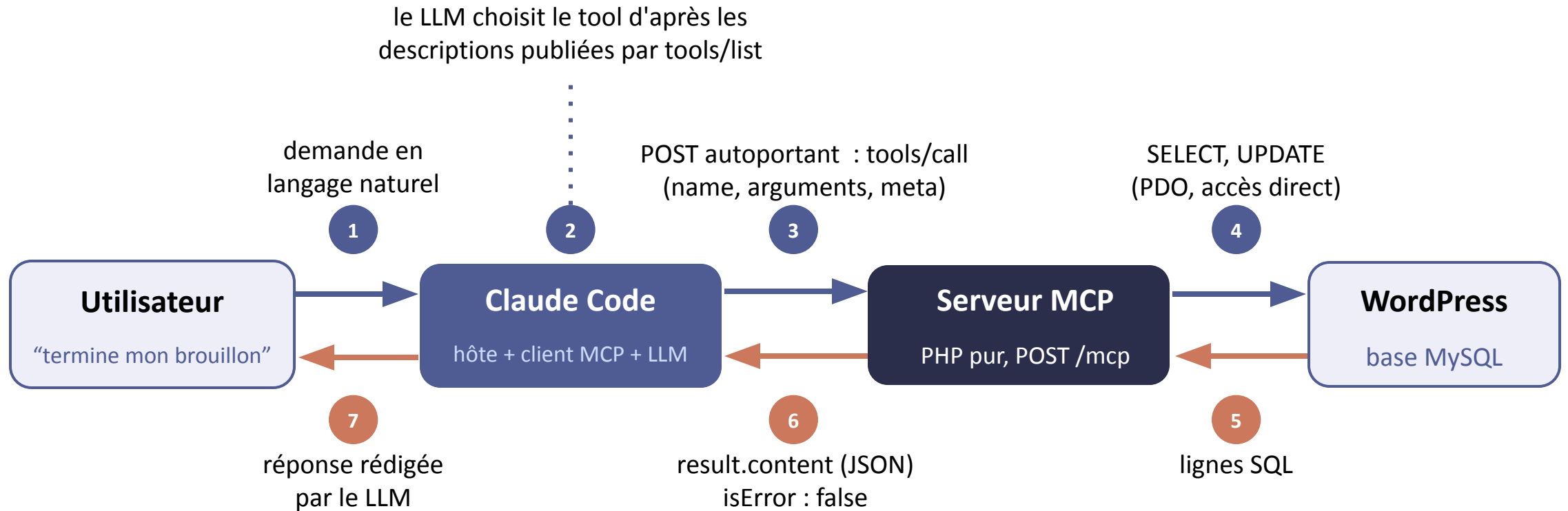
```
JSON-RPC.json

// La discovery. Le LLM choisit ses tools d'après ces descriptions.
// ttlMs et cacheScope : le client peut mettre la liste en cache.
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "tools": [
      {
        "name": "update_post",
        "description": "Remplace le contenu d'un article (pas le titre ni le slug).",
        "inputSchema": {
          "type": "object",
          "properties": {
            "id": {"type": "integer", "description": "Identifiant de l'article"},
            "content": {"type": "string", "description": "Nouveau contenu complet"},
            "status": {"type": "string", "enum": ["draft", "publish"]}
          },
          "required": ["id", "content"]
        }
      }
    ],
    "resultType": "complete",
    "ttlMs": 300000,
    "cacheScope": "public"
  }
}
```

À retenir

- La description est lue par le LLM : c'est du prompt engineering déguisé.
- inputSchema : du JSON Schema standard.
- ttlMs + cacheScope : le client peut mettre la liste en cache, comme un Cache-Control HTTP.
- Bien décrire = le modèle choisit bien.
Mal décrire = il ignore votre tool.

Le flux complet d'un appel



La boucle 2 à 6 peut se répéter : le LLM enchaîne plusieurs tool calls
(search_posts, puis get_post, puis update_post)

Pourquoi ce serveur n'expose que des tools

1

Le langage naturel suffit à déclencher les tools

« *Affiche un résumé de l'article sur les courgettes* » → `search_posts`, puis `get_post`.

2

Resources et prompts exigent un geste explicite

Dans les faits, on préfère taper « *Résume l'article sur les courgettes* » que « *Résume @blog:wp://post/101* »

3

Représentatif de l'écosystème

La grande majorité des serveurs MCP en production n'exposent que des tools.

4

Le protocole reste complet

Dans notre implémentation, `resources/list` et `prompts/list` retournent des listes vides.

3

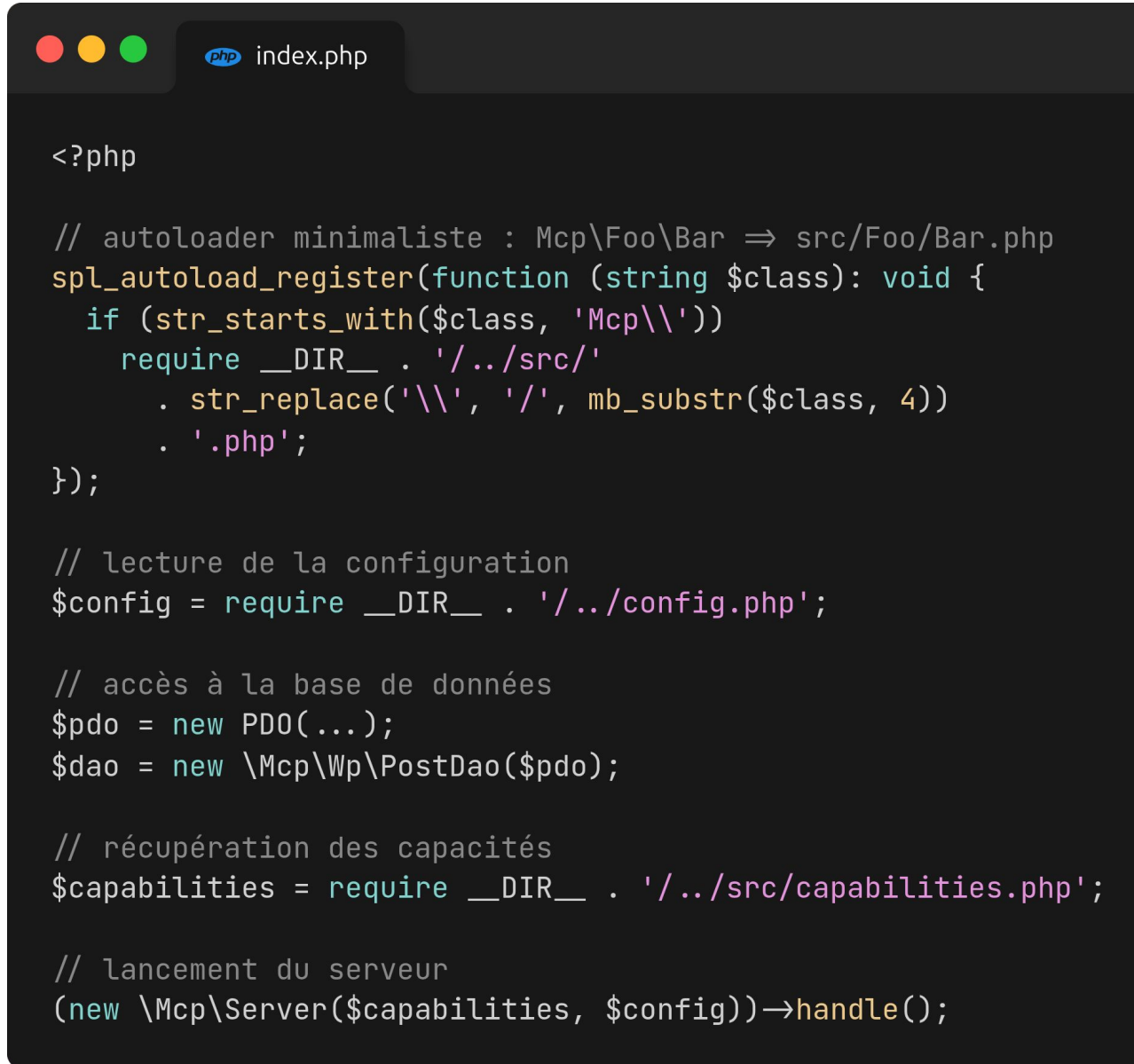
Construction du serveur

Transport HTTP

Dispatcher JSON-RPC

Discovery : le cœur, en moins de 300 lignes

Instantiation du serveur



```
<?php

// autoloader minimaliste : Mcp\Foo\Bar => src/Foo/Bar.php
spl_autoload_register(function (string $class): void {
    if (str_starts_with($class, 'Mcp\\'))
        require __DIR__ . '/../src/'
            . str_replace('\\', '/', mb_substr($class, 4))
            . '.php';
});

// lecture de la configuration
$config = require __DIR__ . '/../config.php';

// accès à la base de données
$pdo = new PDO(...);
$dao = new \Mcp\Wp\PostDao($pdo);

// récupération des capacités
$capabilities = require __DIR__ . '/../src/capabilities.php';

// lancement du serveur
(new \Mcp\Server($capabilities, $config))>handle();
```

À retenir

- Pour lancer le serveur :
php -S 127.0.0.1:8080 public/index.php
- Ou : make mcp

Le transport tient dans une classe

```
Server.php

<?php
// Le transport Streamable HTTP, version minimale
final class Server {
    private const ENDPOINT = '/mcp';

    public function handle(): void {
        $path = parse_url($_SERVER['REQUEST_URI'] ?? '/', PHP_URL_PATH);
        if ($path !== self::ENDPOINT) {
            $this->respond(404, null);
        } elseif (($_SERVER['REQUEST_METHOD'] ?? '') !== 'POST') {
            header('Allow: POST');
            $this->respond(405, null);
        } elseif (!$this->originAllowed()) {
            $this->respond(403, null);
        } else {
            $this->handleMessage(
                (string) file_get_contents('php://input')
            );
        }
    }

    // ...
}
```

À retenir

- GET et DELETE → 405 : c'étaient les mécanismes des révisions à session.
- Contrôle de l'en-tête Origin : protection contre le DNS rebinding.
- La méthode `handleMessage()` fait quelques vérifications puis appelle `dispatch()`

La couche JSON-RPC

```
<?php
// La couche JSON-RPC tient dans une classe statique
class JsonRpc {
    public static function decode(string $body): ?array {
        $msg = json_decode($body, true);
        return is_array($msg) ? $msg : null;
    }

    public static function isValid(array $msg): bool {
        return ($msg['jsonrpc'] ?? null) === '2.0' &&
            is_string($msg['method'] ?? null);
    }

    public static function isNotification(array $msg): bool {
        return !array_key_exists('id', $msg);
    }

    public static function success(mixed $id, array $result): array {
        return [
            'jsonrpc' => '2.0',
            'id'      => $id,
            'result'  => $result,
        ];
    }
}
```

À retenir

- Une classe statique : décoder, valider, fabriquer les réponses.
- json_decode + trois vérifications : la "couche protocole" est là.
- La détection des notifications : absence de la clé id.

Le dispatch : un match et c'est tout

```
<?php
class Server {
    // ...
    // Tout le protocole MCP tient dans un match
    private function dispatch(string $method, array $params): array {
        return match ($method) {
            // discovery
            'server/discover'      => $this->discover(),
            // tools
            'tools/list'           => ['tools' => $this->toolList()]
                                   + $this->cacheable(),
            'tools/call'           => $this->callTool($params),
            // resources
            'resources/list'       => ['resources' => $this->resourceList(false)]
                                   + $this->cacheable(),
            'resources/templates/list' => ['resourceTemplates' => $this->resourceList(true)]
                                   + $this->cacheable(),
            'resources/read'       => $this->readResource($params),
            // prompts
            'prompts/list'         => ['prompts' => $this->promptList()]
                                   + $this->cacheable(),
            'prompts/get'          => $this->getPrompt($params),
            // erreur
            default => throw new \BadFunctionCallException("Méthode inconnue : $method"),
        };
    }
    // ...
}
```

À retenir

- Huit méthodes MCP suffisent pour un serveur complet.
- Méthode inconnue → 404 + erreur -32601, paramètres invalides → -32602.

server/discover : la carte de visite du serveur

```
<?php
// La carte de visite du serveur, et la forme des réponses.
class Server {
    // ...
    // 1. server/discover : versions, capacités et identité, en un appel.
    private function discover(): array {
        return [
            'supportedVersions' => ['2026-07-28'],
            'capabilities'       => ['tools' => new \stdClass()],
            'instructions'       => 'Serveur branché sur un blog WordPress.',
            'ttlMs'              => 3600000,
            'cacheScope'         => 'public',
        ];
    }
    // ...
}

// 2. Les en-têtes reprennent méthode et nom : on vérifie qu'ils collent au corps.
if (($_SERVER['HTTP_MCP_METHOD'] ?? '') !== $msg['method'])
    return 'Mcp-Method absent ou différent de la méthode du corps.';

// 3. Chaque résultat est décoré : type de résultat + identité du serveur.
$result += [
    'resultType' => 'complete',
    '_meta'      => ['io.modelcontextprotocol/serverInfo' => $this->config['server']]
];
```

À retenir

- Le client demande versions, capacités et identité en un seul appel.
- Le serveur vérifie que les en-têtes Mcp-* collent au corps de la requête.
- Chaque résultat est décoré : resultType + serverInfo dans _meta.

Tableau des capacités du serveur

```
<?php
// La carte du serveur, un simple tableau associatif.
// Les parties descriptives partent telles quelles dans la discovery ;
// les closures sont exécutées à l'appel.
return [
    'tools' => [
        'get_post' => [
            'description' => "Retourne un article complet :
                             contenu, statut, dates, catégories et tags.",
            'inputSchema' => [
                'type' => 'object',
                'properties' => [
                    'id' => [
                        'type' => 'integer',
                        'description' => "Identifiant de l'article"
                    ],
                ],
                'required' => ['id']
            ],
            'annotations' => ['readOnlyHint' => true],
            'handler' => function (array $args) use ($dao): array {
                return $dao->getPost($args['id']);
            },
        ],
        // ... search_posts, update_post
    ],
];
```

À retenir

- Tout ce que le serveur expose, déclaré dans un seul tableau associatif.
- Les parties descriptives partent telles quelles dans la discovery ; les closures sont exécutées à l'appel.
- Ajouter une primitive = ajouter une entrée. Dans un vrai projet : des classes et des interfaces.

Appel d'un *tool* et gestion des erreurs

```
<?php
class Server {
    // ...
    // Exécution d'un tool.
    // Appel du handler, sérialisation de sa réponse (le protocole veut
    // du texte) et gestion des erreurs.
    private function callTool(array $params): array {
        $tool = $this->capabilities['tools'][$params['name']] ?? '';
        ?? throw new \InvalidArgumentException('Tool inconnu.');
```

\$handler = \$tool['handler'];

```
        try {
            // Le handler rend des données ; le protocole veut du texte dans content.
            $result = $handler($params['arguments']);
            $textResult = json_encode($result ?? [], JSON_UNESCAPED_UNICODE);
            return [
                'content' => [[
                    'type' => 'text',
                    'text' => $textResult,
                ]],
                'isError' => false,
            ];
        } catch (\Throwable $e) {
            // Erreur d'exécution : signalée DANS le résultat (isError), pas au
            // niveau JSON-RPC, pour que le LLM puisse la lire et réagir.
            return [
                'content' => [['type' => 'text',
                    'text' => 'Erreur : ' . $e->getMessage()]],
                'isError' => true
            ];
        }
    }
    // ...
}
```

À retenir

- Erreur protocolaire (tool inconnu) → error JSON-RPC, pour le client.
- Erreur métier (article introuvable) → isError: true, pour le LLM.
- Le LLM lit le message d'erreur et corrige son tir tout seul.

4

Intégration WordPress

3 tools :

```
get_post  
update_post  
search_posts
```

PDO direct, sans wp-load.php

Lire un article

```
McP/Wp/PostDao.php

<?php
class PostDao {
    // ...
    // lire un article
    public function getPost(int $id): array {
        $stmt = $this->pdo->prepare(
            "SELECT ID, post_author, post_title, post_name, post_status,
            post_date, post_modified, post_content
            FROM wp_posts
            WHERE ID = ?
            AND post_type = 'post'"
        );
        $stmt->execute([$id]);
        if (($post = $stmt->fetch()) === false)
            throw new \DomainException("Article $id introuvable.");
        // on ajoute catégories et tags au résultat
        return $post + $this->terms($id);
    }
    // ...
}
```

À retenir

- C'est le *tool* que le LLM appelle le plus : il lit avant d'écrire.
- Article introuvable → exception, que le serveur transforme en `isError` pour le modèle.

Mise à jour d'un article

```
capabilities.php

<?php
return ['tools' => [
    // ...
    // Déclarer un tool. La description est lue par le LLM (du prompt
    // engineering déguisé), le schéma cadre les paramètres, les annotations
    // aident le client à décider, la closure exécute et rend des DONNÉES.
    'update_post' => [
        'description' => "Remplace le contenu d'un article (le titre et le slug ne changent pas).
        Peut aussi changer son statut (draft ou publish).",
        'inputSchema' => [
            'type' => 'object',
            'properties' => [
                'id' => ['type' => 'integer', 'description' => "Identifiant de l'article"],
                'content' => ['type' => 'string', 'description' => "Nouveau contenu complet"],
                'status' => ['type' => 'string', 'enum' => ['draft', 'publish']],
            ],
            'required' => ['id', 'content']
        ],
        // hints : écrase du contenu, mais rejouable sans dégât
        'annotations' => [
            'readOnlyHint' => false,
            'destructiveHint' => true,
            'idempotentHint' => true
        ],
        'handler' => function (array $args) use ($dao): array {
            if (!isset($args['id'], $args['content']))
                throw new \InvalidArgumentException("Paramètres id et content requis.");
            return $dao->updatePost($args['id'], $args['content'], $args['status'] ?? null);
        },
    ],
],
```

À retenir

- Description, inputSchema, handler : ce que le LLM lit, ce qu'il envoie, ce qu'on exécute.
- Les annotations (readOnlyHint, destructiveHint, idempotentHint) aident le client à décider : auto-approuver une lecture, confirmer une écriture.
- Ce sont des hints déclaratifs, non vérifiés : les vrais garde-fous restent côté serveur (droits SQL).

Écrire dans WordPress sans WordPress

```
<?php
class PostDao {
    // WordPress sans WordPress. PDO direct, pas de wp-load.php.
    public function updatePost(int $id, string $content,
                               ?string $status = null): array {
        // vérifie l'existence du post, lève une exception sinon
        $this->getPost($id);
        // prépare la requête SQL
        $sql = "UPDATE wp_posts
                SET post_content = ?,
                    post_modified = ?,
                    post_modified_gmt = ?"
                . ($status !== null ? ', post_status = ?' : '')
                . ' WHERE ID = ?';
        $params = [$content, date('Y-m-d H:i:s'), gmdate('Y-m-d H:i:s')];
        if ($status !== null)
            $params[] = $status;
        $params[] = $id;
        // exécute la requête SQL
        $this->pdo->prepare($sql)->execute($params);
        // résultat
        return [
            'id'      => $id,
            'updated' => true,
            'status'  => $status ?? 'unchanged',
        ];
    }
    // ...
}
```

À retenir

- post_modified mis à jour à la main : c'est le code PHP de WP qui le fait d'habitude.
- On ne touche que post_content (+ statut) : titre et slug intacts.

Recherche d'articles

```
capabilities.php

<?php
return ['tools' => [
    // Tool de recherche d'articles.
    'search_posts' => [
        'description' => "Liste les articles du blog, du plus récent au plus vieux.
        Sans paramètre : les derniers articles.
        Avec status=draft : les brouillons.
        Avec query : recherche dans les titres et les contenus.",
        'inputSchema' => [
            'type' => 'object',
            'properties' => [
                'query' => ['type' => 'string',
                    'description' => "Texte recherché (optionnel)"],
                'status' => ['type' => 'string',
                    'description' => "Filtrer par statut (défaut : any)",
                    'enum' => ['publish', 'draft', 'any']],
                'limit' => ['type' => 'integer',
                    'description' => "Nombre maximum de résultats (défaut : 20)"],
            ],
        ],
        'annotations' => ['readOnlyHint' => true],
        'handler' => function (array $args) use ($dao): array {
            $args['query'] ??= null;
            $args['status'] ??= 'any';
            $args['limit'] ??= 20;
            return $dao->searchPosts($args['query'], $args['status'], $args['limit']);
        },
    ],
],
```

À retenir

- Description, inputSchema, handler : ce que le LLM lit, ce qu'il envoie, ce qu'on exécute.
- Pas de required : tous les paramètres sont optionnels.

Recherche dans WordPress

```
McP/Wp/PostDao.php

<?php
class PostDao {
    // ...
    /**
     * Recherche et liste les articles, du plus récent au plus ancien.
     * Sans argument : les derniers articles (publiés et brouillons).
     * $status filtre par statut ('any' : tous), $query cherche dans les
     * titres et les contenus (LIKE naïf, suffisant pour la démo).
     */
    public function searchPosts(?string $query = null, string $status = 'any',
                                int $limit = 20): array {
        $sql = "SELECT ID, post_title, post_status, post_date, post_modified,
                     SUBSTRING(post_content, 1, 200) AS content_start
                FROM wp_posts
                WHERE post_type = 'post'";
        $params = [];
        if ($status == 'any') {
            $sql .= " AND post_status IN ('publish', 'draft')";
        } else {
            $sql .= " AND post_status = ?";
            $params[] = $status;
        }
        if ($query !== null && $query !== '') {
            $like = '%' . addslashes($query, '%_\\') . '%';
            $sql .= " AND (post_title LIKE ? OR post_content LIKE ?)";
            array_push($params, $like, $like);
        }
        $sql .= " ORDER BY post_date DESC LIMIT " . max(1, min(50, $limit));
        $stmt = $this->pdo->prepare($sql);
        $stmt->execute($params);
        return $stmt->fetchAll();
    }
}
```

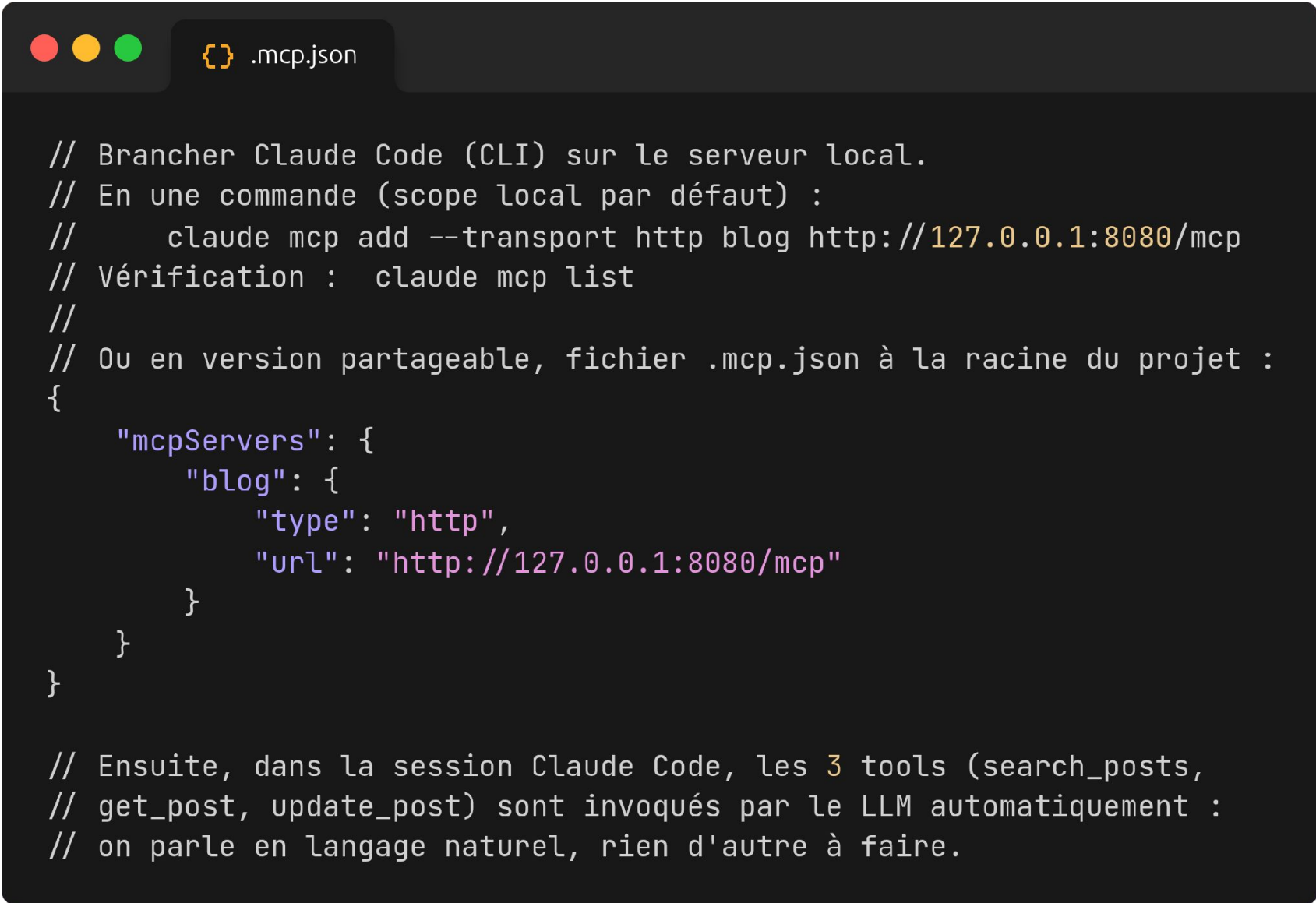
À retenir

- post_modified mis à jour à la main : c'est le code PHP de WP qui le fait d'habitude.
- On ne touche que post_content (+ statut) : titre et slug intacts.

5

Démo live

Branchement et programme



```
.mcp.json

// Brancher Claude Code (CLI) sur le serveur local.
// En une commande (scope local par défaut) :
//   claude mcp add --transport http blog http://127.0.0.1:8080/mcp
// Vérification : claude mcp list
//
// Ou en version partageable, fichier .mcp.json à la racine du projet :
{
  "mcpServers": {
    "blog": {
      "type": "http",
      "url": "http://127.0.0.1:8080/mcp"
    }
  }
}

// Ensuite, dans la session Claude Code, les 3 tools (search_posts,
// get_post, update_post) sont invoqués par le LLM automatiquement :
// on parle en langage naturel, rien d'autre à faire.
```

6

Limites, pièges, et la suite

Ce qu'on n'a pas implémenté
Les critiques du protocole
Le SDK officiel

Ce qu'on a laissé de côté (en connaissance de cause)



Streaming SSE

Optionnels dans le transport. Répondre en JSON simple est conforme.



Multi Round-Trip Requests (`resultType` de type `input_required`)

Le nouveau mécanisme d'interaction serveur→client. Nos tools n'en ont pas besoin.



OAuth 2.1

Obligatoire pour un serveur HTTP exposé publiquement. En local : Bearer token, voire rien.



Pièges : caches et écritures

Pas de cache WordPress pendant la démo. Un LLM écrit en base : droits SQL minimaux, GRANT par colonne, jamais en prod.

Le protocole a aussi ses détracteurs

PERPLEXITY

mars 2026 : de simples API REST et des CLI

Abandonne MCP en interne et revient aux outils classiques : API REST et lignes de commande.
Motifs : les schémas de tools mangeaient jusqu'à 72 % de la fenêtre de contexte avant la première requête, et l'authentification créait trop de frictions.

Nuance : leur serveur MCP public reste maintenu.

CLOUDFLARE

« Code Mode » : le problème des catalogues géants

Exposer ~2 500 endpoints d'API en tools MCP :
~1,17 million de tokens de définitions. Leur réponse : 2 tools (search, execute) et du code généré, ~1 000 tokens.

*Nuance : construit sur les schémas MCP.
L'orchestration change, pas le protocole.*

La critique vise les catalogues géants de tools, pas le protocole.

La suite : le SDK officiel mcp/sdk

```
<?php
// Le même tool avec le SDK officiel mcp/sdk (en stabilisation,
// PHP Foundation + Symfony). Déclaratif par attributs, schéma déduit de la
// signature. À vérifier : l'API bouge encore.
use Mcp\Capability\Attribute\McpTool;
use Mcp\Server;

class PostTools
{
    // ...

    #[McpTool(
        name: 'get_post',
        description: 'Retourne un article complet.'
    )]
    public function getPost(int $id): array
    {
        return $this->repository->getPost($id);
    }
}

Server::builder()
    ->setServerInfo('wp-mcp-server', '1.0.0')
    ->addTool([PostTools::class, 'getPost'])
    ->build()
    ->run();
```

À retenir

- Attributs PHP 8, schéma déduit de la signature : moins de plomberie.
- Créé par la PHP Foundation + Symfony
- Encore instable.
- Vous savez maintenant exactement ce qu'il fait sous le capot.

Merci !

Le serveur, le dump WordPress de démo et ces slides :

github.com/Amaury/mcp-forumphp2026

geek-directeur-technique.com/forumphp2026

Amaury Bouchard

 @geekcto

 @amaury@phpc.social

 geek-directeur-technique.com